AI Agent Evaluation Metrics

A literature review with definitions, formulas, ranges, thresholds and worked examples

Primary reference: H. Roitman, *The Hitchhiker's Guide to Agentic AI: From Foundations to Systems*, arXiv:2606.24937 (submitted 22 Jun 2026; revised 27 Jul 2026).

Prepared: 7 October 2026

Read this first — what was and was not verified. The Hitchhiker's Guide is a 700+ page book. In this session its abstract and **complete table of contents** loaded (versions v1.3 and v1.4), but the **body text of its evaluation chapters did not load**. Therefore: (1) the guide is used as the structural reference — every metric lists the guide's relevant sections, taken from its table of contents; (2) I do **not** claim what the guide's text says about any formula or threshold; (3) formulas are the standard definitions from the original papers; (4) numbers quoted from papers are tagged **[verified]** (abstract fetched this session: τ-bench, WebArena, RAGAS, AgentDojo) or **[from memory]** (arXiv IDs and details not re-checked — please confirm before citing). Section numbers follow the v1.4 table of contents where the two versions differ.

Thresholds. Where the literature gives no universal pass/fail value, this review says so explicitly ("task-dependent") and offers an **engineering recommendation**, labelled as such. A score such as 90% is not automatically good: it depends on the task, the human baseline, the cost of errors and the sample size.

1. Summary of findings

• **Headline success is low on realistic benchmarks.** WebArena's best GPT-4 agent reached 14.41% task success against 78.24% for humans, and τ-bench reports state-of-the-art function-calling agents below 50% [both verified]. These are 2023–2024 results; current systems score higher, so check the live leaderboards.

• **Capability and reliability are different quantities.** pass@k asks whether any of k tries works; pass^k asks whether all k tries work. τ-bench reports pass^8 below 25% in its retail domain for a leading agent [verified]. For anything user-facing, report pass^k or consistency, not only pass@k.

• **No agent metric has a universal threshold.** Published work reports scores relative to baselines and human performance. Thresholds must come from your risk, SLA and human baseline. The only conventional cut-offs in this review are interpretation bands such as Landis–Koch for κ.

• **Always pair metrics.** Success with cost and latency; ASR with utility under attack; faithfulness with context recall; judge scores with human agreement; precision with recall.

• **Uncertainty matters.** With 50 tasks, 74% success has a 95% interval of roughly 60%–84% (Section 3, TSR example). Report intervals and repeat runs.

2. How the Hitchhiker's Guide covers agent evaluation (from its table of contents)

Topic	Sections in the guide (v1.4 numbering, TOC verified)
Ranking & preference	§14.4 Metrics for Ranking Tasks: ELO, Bradley–Terry, TrueSkill, Win Rate with CIs, Chatbot Arena methodology
Generation & code metrics	§14.5 incl. §14.5.6 Pass@k for Code; §14.5.7 Exact Match and F1
Agentic metrics	§14.6: Task Success Rate, Trajectory Efficiency, Tool-Use Accuracy, Multi-Step Reasoning Accuracy, SWE-bench and WebArena methodology
LLM-as-Judge	§14.7: setup and prompts, position-bias mitigation, multi-judge panels, agreement metrics, G-Eval, JEV-as-a-Judge
Human evaluation	§14.2: annotation pipelines, inter-annotator agreement (Cohen's, Fleiss' κ), guidelines, crowdsourcing vs experts
Pitfalls	§14.8: contamination, overfitting, Goodhart's law, reasoning collapse under reformulation
RAG / grounding	§16.8: retrieval metrics, faithfulness, answer relevance, context precision/recall, RAGAs
Memory	§17.9: evaluation dimensions, benchmarks, memory-level, downstream and operational metrics
Environments & harness	Ch. 22: evaluation harness design, deterministic vs stochastic environments, production-grade evaluation of non-deterministic agents, trace-level regression tests, emerging 2026 benchmarks
Safety & security	§1.16 Hallucination Detection; §1.17.5 safety evaluation; §21.5 Agentic Security and Red Teaming
Cost, latency, testing	§18.8 (latency, cost, evaluation in production); §27.5 Agent Testing (golden trajectories, cost and latency testing)
Quick references	§30.14 Evaluation Metrics; §30.22 Agent Evaluation Metrics; §30.23 Key Agentic Benchmarks (titles only; contents unread)

No dedicated sections appear in the table of contents for tool-call precision/recall/F1, argument accuracy, plan adherence, or pass^k. I treat these using their original sources and say so in each entry.

3. Metric-by-metric review

Each entry lists: definition, formula, range, direction, good and bad ranges, recommended threshold, worked example (computed by script), advantages and limitations, and the paper/benchmark where it is used. “[verified]” = abstract checked this session; “[from memory]” = not re-checked.

Task success & goal completion

1. Task Success Rate (TSR)	
Definition	Fraction of tasks for which the agent’s final outcome satisfies a programmatic or annotated success check (e.g., final database state equals the goal state, tests pass, expected page state).
Formula	TSR = (# tasks with success check = true) / (# tasks)
Range	0 to 1 (0%–100%)
Direction	Higher is better
Good range	Close to or above the human baseline on the same benchmark; no universal number
Bad range	Far below human baseline or below a trivial baseline
Recommended threshold	No universal threshold (task-dependent). Engineering recommendation: set pass/fail from the human baseline or business SLA, with the lower CI bound (not the point estimate) above it.
Example calculation	37 of 50 tasks reach the goal state → 37/50 = 0.74 (74%). Wilson 95% CI = [0.604, 0.841], i.e. with only 50 tasks the true rate could plausibly be anywhere from ~60% to ~84%.
Advantages	Simple, end-to-end, hard to game when checks are state-based; maps directly to user value.
Limitations	Binary; hides partial progress, process quality, cost and safety; sensitive to checker quality and sample size; single-run values ignore variance.
Used in (paper / benchmark)	WebArena (best GPT-4 agent 14.41% vs human 78.24%) [verified]; τ-bench (gpt-4o <50%) [verified]; SWE-bench; AgentBench; GAIA
In the Hitchhiker’s Guide	Hitchhiker §14.6.1 Task Success Rate; §14.6.5 SWE-bench; §14.6.6 WebArena (TOC verified)

2. Progress Rate / Partial Goal Completion	
Definition	Measures how far the agent got toward the goal using annotated sub-goals/milestones, instead of only final pass/fail.
Formula	PR = (1/N) Σ_i max_t (matched sub-goals at step t / total sub-goals of task i)
Range	0 to 1
Direction	Higher is better
Good range	High and close to TSR (if PR ≫ TSR the agent gets close but does not finish)
Bad range	Low PR: agent stalls early
Recommended threshold	No published universal threshold (task-dependent); engineering recommendation: use as a diagnostic, not a gate.
Example calculation	Three tasks, best matched sub-goals: 2/4, 4/4, 1/5 → (0.50 + 1.00 + 0.20)/3 = 0.567.
Advantages	More sensitive than TSR for hard tasks; helps locate where agents fail.
Limitations	Needs sub-goal annotation; sub-goal definitions are subjective; partial credit can mask unusable end results.
Used in (paper / benchmark)	AgentBoard (arXiv:2401.13178) [from memory, not re-verified]
In the Hitchhiker’s Guide	Hitchhiker §14.6 (agentic metrics) — the TOC lists Task Success Rate; whether PR is defined in the body text could not be checked

Planning & reasoning

3. Plan Validity / Planning Accuracy	
Definition	A plan is valid if each action's preconditions hold and the goal state is reached when the plan is executed (checked by a validator).
Formula	Plan accuracy = (# valid goal-reaching plans) / (# plans generated)
Range	0 to 1
Direction	Higher is better
Good range	Task-dependent; classical planning benchmarks have shown low LLM scores without tools
Bad range	Near 0 on standard planning tasks
Recommended threshold	No universal threshold (task-dependent). Engineering recommendation: require validator-confirmed validity for any plan that will be executed automatically.
Example calculation	24 generated plans, 18 are valid and reach the goal → 18/24 = 0.75.
Advantages	Objective when a symbolic validator exists; separates planning from execution.
Limitations	Needs formal domain model; does not measure plan optimality; synthetic domains may not transfer to open-ended agent tasks.
Used in (paper / benchmark)	PlanBench (arXiv:2206.10498) [from memory]; ReAct (arXiv:2210.03629) [from memory]
In the Hitchhiker's Guide	Hitchhiker §21.2.2 Planning Agents; §14.6.4 Multi-Step Reasoning Accuracy; §18.5.2 Plan-and-Execute (TOC verified)

Tool use

4. Tool-Selection Accuracy	
Definition	Share of tool-call decisions where the selected tool (function name) matches the reference tool, including correctly choosing no tool when none is needed.
Formula	Tool accuracy = (# correct tool selections) / (# tool-call decisions)
Range	0 to 1
Direction	Higher is better
Good range	Task-dependent; higher with fewer, well-described tools
Bad range	Drops sharply as tool library grows
Recommended threshold	No universal threshold (task-dependent). Engineering recommendation: for write/destructive tools require ≥ the success level you would accept for the action itself, otherwise add confirmation.
Example calculation	50 calls, 45 invoke the correct tool → 45/50 = 0.90.
Advantages	Cheap to compute from logs; isolates tool-routing errors.
Limitations	Several tools may be acceptable; ignores arguments and ordering; reference calls can be incomplete.
Used in (paper / benchmark)	Berkeley Function-Calling Leaderboard (Gorilla, arXiv:2305.15334) [from memory]; ToolLLM/ToolBench (arXiv:2307.16789) [from memory]
In the Hitchhiker's Guide	Hitchhiker §14.6.3 Tool-Use Accuracy; §18.4.2 Tool Selection and Routing (TOC verified)

5. Tool-Call Precision, Recall & F1	
Definition	Treats called tools as a predicted set and reference tools as the gold set. Precision penalises unnecessary/wrong calls; recall penalises missed calls.
Formula	P = TP/(TP+FP); R = TP/(TP+FN); F1 = 2PR/(P+R)
Range	Each 0 to 1
Direction	Higher is better
Good range	task-dependent; read P and R separately
Bad range	Low P = noisy/over-calling; low R = missing required steps
Recommended threshold	No universal threshold (task-dependent). Engineering recommendation: set recall high for safety-critical steps and precision high where calls cost money or have side effects.
Example calculation	Gold tools {search, calc, email}; agent called {search, calc, weather}. TP=2, FP=1, FN=1 → P = 2/3 = 0.667, R = 2/3 = 0.667, F1 = 0.667.
Advantages	Familiar IR metric; separates over-calling from under-calling; works for unordered tool sets.
Limitations	Ignores order, repetition and arguments; needs a gold set; multiple valid solutions penalise correct alternatives.
Used in (paper / benchmark)	Standard IR definition applied to tool calls; used in tool-use benchmarks such as ToolBench/API-Bank-style evaluations [from memory]
In the Hitchhiker's Guide	Hitchhiker §14.5.7 Exact Match and F1; §14.6.3 Tool-Use Accuracy. No dedicated tool-call-F1 section in the TOC; body text unread

6. Argument / Parameter Accuracy	
Definition	Compares argument names and values (after normalisation) with the reference call, at parameter level or whole-call exact-match level; BFCL also checks via AST matching and execution.
Formula	Param acc = (# correct argument values)/(# required argument values); Call EM = (# fully correct calls)/(# calls)
Range	0 to 1
Direction	Higher is better
Good range	task-dependent; call-level exact match is the stricter number
Bad range	High tool accuracy but low argument accuracy signals 'right tool, wrong inputs'
Recommended threshold	No universal threshold (task-dependent). Engineering recommendation: schema-validity (valid JSON/types) should be ≈100%; semantic correctness thresholds depend on the action's risk.
Example calculation	4 calls need 10 argument values in total; 8 are correct → parameter accuracy = 0.80. Only 2 of the 4 calls are fully correct → exact-call match = 0.50. (The two numbers differ; report both.)
Advantages	Catches the most common real-world failure (wrong values); automatable by schema/AST/execution.
Limitations	Equivalent values can be written differently (false negatives); reference calls may be one of many valid ones.
Used in (paper / benchmark)	Berkeley Function-Calling Leaderboard (Gorilla, arXiv:2305.15334) [from memory]
In the Hitchhiker's Guide	Hitchhiker §14.6.3 Tool-Use Accuracy; §18.4.1 Tool Definition Schemas; §1.12.11 Constrained Decoding (TOC verified)

Planning & reasoning

7. Plan Quality & Plan Adherence	
Definition	Quality: rubric/LLM-judge score or optimality ratio against a reference plan. Adherence: share of planned steps executed as planned (in order).
Formula	Adherence = (# plan steps executed as planned)/(# plan steps); Optimality = cost(best plan)/cost(agent plan)
Range	0 to 1 (optimality ≤ 1 if best is known)
Direction	Higher is better
Good range	task-dependent
Bad range	Low adherence with high success = plan is decorative; high adherence with low success = bad plan
Recommended threshold	No universal threshold (task-dependent); engineering recommendation: track as diagnostics, gate on TSR and safety.
Example calculation	Plan has 8 steps; 6 are executed as planned and in order → adherence = 6/8 = 0.75. Plan optimality: best known plan costs 5 actions, agent's plan costs 7 → 5/7 = 0.714.
Advantages	Exposes planning-vs-execution failures; supports debugging of plan-and-execute agents.
Limitations	Re-planning is often desirable, so low adherence is not always bad; needs reference plans or judge.
Used in (paper / benchmark)	Plan-and-Execute / ReAct-family agent evaluations (arXiv:2210.03629) [from memory]; no single canonical benchmark definition
In the Hitchhiker's Guide	Hitchhiker §18.5.2 Plan-and-Execute; §21.2.2 Planning Agents (TOC verified); body text unread

Trajectory & process

8. Trajectory / Process Evaluation	
Definition	Scores each step or the whole path: step-level correctness, trajectory match (exact, in-order, any-order) against a reference, or learned process-reward-model (PRM) scores.
Formula	Step accuracy = (# correct steps)/(# steps); in-order match: reference actions appear as an ordered subsequence
Range	0 to 1
Direction	Higher is better
Good range	task-dependent
Bad range	Correct final answers reached by lucky/unsafe paths
Recommended threshold	No universal threshold (task-dependent); engineering recommendation: use golden-trajectory regression tests to flag changes rather than an absolute number.
Example calculation	Agent trajectory has 9 steps; a step checker marks 7 as correct/useful → step accuracy = 7/9 = 0.778. Under in-order matching against a 5-step reference, all 5 reference actions appearing in order gives recall 1.0 but precision 5/9 = 0.556.
Advantages	Finds right-answer-wrong-reason cases; supports regression testing and credit assignment.
Limitations	Needs references or step labels; multiple valid paths; step judgments (human/LLM) are costly or noisy.
Used in (paper / benchmark)	AgentBench (arXiv:2308.03688) [from memory]; process reward models (Lightman et al., 'Let's Verify Step by Step') [from memory]
In the Hitchhiker's Guide	Hitchhiker §9.4 Process vs Outcome Reward Models; §27.5.3 Regression Testing with Golden Trajectories; §22.11.3 Trace-Level Regression Testing; §14.6.4 (TOC verified)

Efficiency

9. Step / Trajectory Efficiency	
Definition	Ratio of the minimum (or reference) number of steps to the steps actually used; alternatively the mean number of steps/turns per task.
Formula	Efficiency = optimal steps / actual steps (or mean steps per task)
Range	(0, 1] when optimum is known; otherwise 1 to ∞ steps
Direction	Higher is better (as ratio); lower is better (as step count)
Good range	Near 1.0 with success unchanged (task-dependent)
Bad range	Much lower than 1: loops, redundant calls
Recommended threshold	No universal threshold (task-dependent); engineering recommendation: set a step budget per task type and alert on loops (repeated identical calls).
Example calculation	Optimal solution takes 6 steps; the agent used 10 → efficiency = 6/10 = 0.60 (40% of its steps were overhead).
Advantages	Correlates with cost and latency; cheap to compute; detects loops.
Limitations	Optimum unknown for open tasks; fewer steps is not always better (verification steps are valuable).
Used in (paper / benchmark)	Efficiency analyses in agent benchmarks (e.g., AgentBench, SWE-agent step limits) [from memory]
In the Hitchhiker's Guide	Hitchhiker §14.6.2 Trajectory Efficiency (TOC verified)

Pass@k & Pass^k

10. Pass@k	
Definition	Unbiased estimator: draw n ≥ k samples per task, count c correct, and estimate the chance at least one of k randomly chosen samples is correct. Measures capability with retries.
Formula	pass@k = E_tasks[1 − C(n−c, k) / C(n, k)]
Range	0 to 1; non-decreasing in k
Direction	Higher is better
Good range	task-dependent; meaningful only if a verifier can pick the correct sample
Bad range	Large gap between pass@1 and pass@k means answers are not reliable one-shot
Recommended threshold	No universal threshold (task-dependent); engineering recommendation: report pass@1 for products without a verifier; use pass@k only when tests/verifiers select the winner.
Example calculation	n = 10 samples, c = 3 correct, k = 5: pass@5 = 1 − C(7,5)/C(10,5) = 1 − 21/252 = 0.9167.
Advantages	Standard for code; low-variance unbiased estimator; shows headroom from sampling.
Limitations	Over-optimistic for agents that act in the real world (you cannot retry side effects); hides inconsistency.
Used in (paper / benchmark)	Chen et al., Codex / HumanEval (arXiv:2107.03374) [from memory]
In the Hitchhiker's Guide	Hitchhiker §14.5.6 Pass@k for Code (TOC verified)

11. Pass^k (pass-hat-k)	
Definition	Reliability metric introduced with τ-bench: a task counts only if the agent succeeds on every one of k trials.
Formula	pass^k = E_tasks[C(c, k) / C(n, k)] (≈ p^k for success prob. p)
Range	0 to 1; non-increasing in k
Direction	Higher is better
Good range	Stays close to pass^1 as k grows (task-dependent)
Bad range	Steep fall with k (τ-bench: pass^8 <25% in retail for gpt-4o) [verified]
Recommended threshold	No universal threshold (task-dependent). Engineering recommendation: choose k as the number of times a user will plausibly repeat the task; mathematically, 99% per-run success is only ≈ 90% at k = 10 (0.99^{10}).
Example calculation	One task, n = 8 trials, c = 6 successes, k = 4: pass^4 = C(6,4)/C(8,4) = 15/70 = 0.2143. The same data give pass@4 = 1.00: an agent that looks perfect under pass@k can be unreliable under pass^k.
Advantages	Captures consistency; exposes brittleness that pass@k hides; fits customer-facing agents.
Limitations	Needs many trials (cost); strongly dependent on k; stochastic user simulators add noise.
Used in (paper / benchmark)	τ-bench (arXiv:2406.12045) [verified]
In the Hitchhiker's Guide	No pass^k section in the TOC (pass@k at §14.5.6); §22.11 'Production-Grade Evaluation of Non-Deterministic Agents' is related; body unread

Reliability

12. Reliability / Run-to-Run Consistency	
Definition	Fraction of tasks with identical outcomes across k repeated runs, or variance of success across seeds. Complements pass^k.
Formula	Consistency = (1/N) Σ_i 1[all k outcomes of task i are equal]
Range	0 to 1
Direction	Higher is better
Good range	task-dependent; for deterministic decoding should be ≈ 1
Bad range	Many tasks flip between pass and fail
Recommended threshold	No universal threshold (task-dependent); engineering recommendation: require consistency (or pass^k) to be reported with every headline success number.
Example calculation	20 tasks × 5 runs each; 14 tasks give the same outcome in all 5 runs → consistency = 14/20 = 0.70.
Advantages	Cheap to compute from repeated runs; reveals non-determinism.
Limitations	A consistently wrong agent scores 1.0 — always read with TSR.
Used in (paper / benchmark)	τ-bench (consistency framing) [verified]; general non-determinism practice
In the Hitchhiker's Guide	Hitchhiker §22.7.1 Deterministic vs. Stochastic Environments; §22.11 Production-Grade Evaluation of Non-Deterministic Agents (TOC verified)

Judges & preference

13. LLM-as-a-Judge (score and agreement)

Definition	A judge model scores answers pointwise (rubric 1–5/1–10), pairwise (A vs B) or reference-guided. The judge itself is evaluated with agreement metrics such as accuracy, Cohen's κ or correlation.
Formula	Agreement $\kappa = (p_o - p_e)/(1 - p_e)$; also judge accuracy vs human labels
Range	κ from −1 to 1; scores per rubric scale
Direction	Higher agreement is better
Good range	$\kappa \approx 0.61$–0.80 or higher against humans (Landis–Koch 'substantial' band)
Bad range	$\kappa < 0.4$, or strong position/verbosity bias
Recommended threshold	No universal threshold. Engineering recommendation: only trust a judge after calibrating on a human-labelled set (e.g., $\kappa \geq 0.6$) and after swapping answer order to control position bias.
Example calculation	Judge and human agree on 85% of 200 items ($p_o = 0.85$); chance agreement from label frequencies is $p_e = 0.50$ → Cohen's $\kappa = (0.85 - 0.50)/(1 - 0.50) = 0.70$.
Advantages	Scalable and cheap; works for open-ended outputs; G-Eval-style rubrics improve alignment with humans.
Limitations	Position, verbosity and self-preference biases; prompt sensitivity; can be gamed; needs periodic human recalibration.
Used in (paper / benchmark)	MT-Bench / Chatbot Arena judge study (Zheng et al., arXiv:2306.05685) [from memory]; G-Eval (arXiv:2303.16634) [from memory]
In the Hitchhiker's Guide	Hitchhiker §14.7 LLM-as-Judge (§14.7.2 Position Bias, §14.7.3 Multi-Judge Panels, §14.7.4 Agreement Metrics, §14.7.5 G-Eval) (TOC verified)

14. Elo / Pairwise Preference (Bradley–Terry)

Definition	Pairwise outcomes are converted to ratings. Elo updates online; Bradley–Terry fits all comparisons at once and gives confidence intervals (e.g., by bootstrap). Win rate is the raw alternative.
Formula	$E_A = 1/(1+10^{(R_B-R_A)/400})$; $R_A' = R_A + K(S_A - E_A)$; BT: $P(A>B)=\sigma(s_A-s_B)$
Range	Win rate 0–1; Elo/BT scores unbounded (relative)
Direction	Higher is better
Good range	Higher than rivals with non-overlapping confidence intervals
Bad range	Overlapping CIs = no demonstrated difference
Recommended threshold	No pass/fail threshold: Elo is a relative ranking, not an absolute quality score. Engineering recommendation: ship only if win rate vs the current system is >50% with the CI excluding 50%.
Example calculation	$R_A = 1200$, $R_B = 1000$ → $E_A = 1/(1+10^{(-200/400)}) = 0.7597$. If A wins ($K = 32$): $R_A' = 1200 + 32 \cdot (1 - 0.7597) = 1207.69$ (+7.69).
Advantages	No absolute scale needed; robust to prompt diversity; large-scale human signal.
Limitations	Reflects voter preferences (style, length); needs many comparisons; ratings only comparable within one pool.
Used in (paper / benchmark)	Chatbot Arena (arXiv:2403.04132) [from memory]; Bradley–Terry model
In the Hitchhiker's Guide	Hitchhiker §14.4.1 ELO Rating System; §14.4.2 Bradley–Terry; §14.4.3 TrueSkill; §14.4.4 Win Rate with CIs; §14.4.5 Chatbot Arena (TOC verified)

Grounding & factuality

15. Faithfulness / Groundedness

Definition	The answer is decomposed into atomic claims; each is checked (usually by an LLM) against the retrieved context. Measures whether the agent stays within its sources.
Formula	Faithfulness = (# claims supported by context)/(# claims in answer)
Range	0 to 1
Direction	Higher is better
Good range	task-dependent; regulated domains need values near 1
Bad range	Many unsupported claims
Recommended threshold	No universal threshold (task-dependent). Engineering recommendation: medical/legal/financial uses should require near-1 plus human review of failures; casual use can accept lower.
Example calculation	Answer is split into 10 claims; 8 are supported by the retrieved context → faithfulness = 8/10 = 0.80.
Advantages	Reference-free (no gold answer needed); pinpoints unsupported claims.
Limitations	Depends on claim extraction and the verifier LLM; a claim can be supported yet the context itself wrong.
Used in (paper / benchmark)	RAGAS (arXiv:2309.15217) [verified]
In the Hitchhiker's Guide	Hitchhiker §16.8.2 Generation Metrics (Faithfulness); §16.8.3 RAGAs Framework (TOC verified)

16. Factual Accuracy (FActScore-style)

Definition	Generation is split into atomic facts, each verified against a trusted knowledge source; also used as simple answer accuracy on QA sets.
Formula	FActScore = (# supported atomic facts)/(# atomic facts)
Range	0 to 1
Direction	Higher is better
Good range	task-dependent
Bad range	Low scores on long-form generation
Recommended threshold	No universal threshold (task-dependent). Engineering recommendation: define per domain and per claim severity.
Example calculation	Biography decomposed into 50 atomic facts; 42 are supported by the reference source → FActScore = 42/50 = 0.84.
Advantages	Fine-grained; works for long outputs; less all-or-nothing than answer-level accuracy.
Limitations	Depends on knowledge-source coverage and claim splitter; costly to verify.
Used in (paper / benchmark)	FActScore (arXiv:2305.14251) [from memory]; TruthfulQA (arXiv:2109.07958) [from memory]
In the Hitchhiker's Guide	Hitchhiker §1.16 Hallucination Detection; §14.5.7 Exact Match and F1 (TOC verified)

Retrieval quality

17. Context Precision & Context Recall	
Definition	Precision: relevant share of retrieved chunks (RAGAS also weights by rank). Recall: share of reference-answer claims that can be attributed to the retrieved context.
Formula	Context precision = (# relevant retrieved)/(# retrieved); Context recall = (# reference claims supported by context)/(# reference claims)
Range	Each 0 to 1
Direction	Higher is better
Good range	task-dependent
Bad range	Low precision wastes context; low recall means the answer cannot be grounded
Recommended threshold	No universal threshold (task-dependent). Engineering recommendation: set a recall floor first (missing evidence cannot be recovered downstream), then tune precision for cost.
Example calculation	5 retrieved chunks, 3 relevant → simple context precision = 3/5 = 0.60. Reference answer has 4 claims, 3 attributable to the retrieved context → context recall = 3/4 = 0.75. (RAGAS's precision variant also weights by rank.)
Advantages	Separates retrieval from generation failures; reference-light.
Limitations	Relevance judgments are often LLM-made; recall needs reference answers; depends on chunking.
Used in (paper / benchmark)	RAGAS (arXiv:2309.15217) [verified]
In the Hitchhiker's Guide	Hitchhiker §16.8.1 Retrieval Metrics (Recall@K, Precision@K, MRR, NDCG@K); §16.8.2 Context Precision and Recall (TOC verified)

Grounding & factuality

18. Hallucination Rate	
Definition	Responses or claims are labelled hallucinated if they contradict or are unsupported by the source/world knowledge; detection by humans, claim checkers or consistency methods.
Formula	Hallucination rate = (# hallucinated responses [or claims])/(# responses [or claims])
Range	0 to 1
Direction	Lower is better
Good range	task-dependent; near 0 for high-stakes domains
Bad range	Any non-trivial rate in safety-critical use
Recommended threshold	No universal threshold (task-dependent). Engineering recommendation: set by harm severity; define the unit (response vs claim) before comparing numbers.
Example calculation	200 responses reviewed; 6 contain at least one unsupported claim → hallucination rate = 6/200 = 0.030 (3.0%).
Advantages	Direct trust measure; easy to communicate.
Limitations	Definition varies (extrinsic vs intrinsic); response-level rates depend on answer length; detectors are imperfect.
Used in (paper / benchmark)	HaluEval (arXiv:2305.11747) [from memory]; SelfCheckGPT (arXiv:2303.08896) [from memory]; TruthfulQA
In the Hitchhiker's Guide	Hitchhiker §1.16 Hallucination Detection (Semantic Entropy, SelfCheckGPT, DoLa) (TOC verified)

Safety & robustness

19. Attack Success Rate (ASR) & Safety Metrics	
Definition	Fraction of attack cases in which the agent performs the attacker’s goal. Agent security benchmarks also report utility with and without attack, because a refuse-everything agent has ASR 0.
Formula	ASR = (# successful attacks)/(# attack attempts); also utility under attack
Range	0 to 1
Direction	Lower is better (ASR); report utility alongside
Good range	As close to 0 as the risk requires (task-dependent)
Bad range	Any success on high-severity actions
Recommended threshold	No universal threshold. Engineering recommendation: set per severity; irreversible or financial actions should target zero observed ASR plus architectural controls, because low sampled ASR is not a guarantee against adaptive attackers.
Example calculation	40 injected-attack scenarios; the agent performs the attacker's action in 3 → ASR = 3/40 = 0.075 (7.5%). Always report utility-under-attack alongside it.
Advantages	Direct, comparable across defenses; AgentDojo ships 97 tasks and 629 security test cases [verified].
Limitations	Depends on the attack set; adaptive attackers find new attacks; ASR alone rewards over-refusal.
Used in (paper / benchmark)	AgentDojo (arXiv:2406.13352) [verified]; InjecAgent (arXiv:2403.02691), HarmBench (arXiv:2402.04249), AgentHarm [from memory]
In the Hitchhiker’s Guide	Hitchhiker §1.17 LLM Safety and Responsible AI (§1.17.5 Evaluation); §21.5 Agentic Security and Adversarial Red Teaming (v1.4 TOC verified)

20. Robustness & Recovery	
Definition	Robustness: drop in success when prompts, tools or environments are perturbed. Recovery: share of injected failures (tool errors, timeouts, bad outputs) from which the agent completes the task.
Formula	Robustness drop = SR_clean − SR_perturbed; Recovery rate = (# recovered failures)/(# injected failures)
Range	Drop: −1 to 1 (typically 0–1); recovery: 0 to 1
Direction	Lower drop / higher recovery is better
Good range	Small drop; high recovery (task-dependent)
Bad range	Large drop under minor rewording (reasoning-collapse behaviour)
Recommended threshold	No universal threshold (task-dependent). Engineering recommendation: define the maximum tolerated drop from your SLA (e.g., relative drop under a stated perturbation set).
Example calculation	Clean success 0.72, success under perturbation 0.58 → absolute drop = 0.14 (relative drop 19.4%). Separately, 12 injected tool failures with 9 recovered → recovery rate = 9/12 = 0.75.
Advantages	Tests behaviour closer to production than clean benchmarks.
Limitations	Results depend on the chosen perturbations/fault models; hard to cover the long tail.
Used in (paper / benchmark)	ToolEmu (arXiv:2309.15817) [from memory]; AgentDojo [verified]; fault-injection practice
In the Hitchhiker’s Guide	Hitchhiker §18.7 Error Handling and Recovery; §14.8.4 Additional Pitfalls; v1.4 adds ‘Reasoning Collapse Under Reformulation’ (TOC verified)

Cost & efficiency

21. Latency	
Definition	End-to-end time (and time to first token) per request/task, reported as percentiles p50/p95/p99 rather than the mean.
Formula	Latency = t_end − t_start; report p50, p95, p99
Range	0 to ∞ (seconds)
Direction	Lower is better
Good range	Within the UX/SLA budget (task-dependent)
Bad range	Long tail (p95/p99) beyond the budget
Recommended threshold	No universal threshold (task-dependent). Engineering recommendation: set a p95 budget by interaction type — interactive chat is judged in seconds, background agents in minutes; usability research often cites ~1 s as flow-preserving and ~10 s as the limit of attention.
Example calculation	Ten end-to-end times (s): [1.8, 2.1, 2.4, 2.6, 3.0, 3.3, 3.9, 4.8, 7.2, 14.5]. Median (p50) = 3.15 s; p95 = 11.21 s (linear interpolation). The mean (4.56 s) hides the slow tail.
Advantages	Directly tied to user experience; simple to measure.
Limitations	Depends on hardware, load, network and tool latency; not comparable across deployments.
Used in (paper / benchmark)	Production monitoring practice; not defined by a single benchmark paper
In the Hitchhiker's Guide	Hitchhiker §18.8.1 Latency Optimization; §16.9.3 Latency Optimization; §27.5.6 Cost and Latency Testing; §11.8 End-to-End Latency Breakdown (TOC verified)

22. Token Usage	
Definition	Total prompt and completion tokens over all steps of a task, often broken down by step, tool output and reasoning tokens.
Formula	Tokens/task = Σ_steps (input tokens + output tokens)
Range	0 to ∞
Direction	Lower is better (at equal success)
Good range	Low at equal success (task-dependent)
Bad range	Growth with each step (context re-reads), loops
Recommended threshold	No universal threshold (task-dependent). Engineering recommendation: set a hard per-task token budget and an alert at e.g. 2× the median.
Example calculation	A 6-step run uses 12,000 input tokens and 2,000 output tokens in total → 14,000 tokens/task. Because every step re-reads context, input tokens usually dominate.
Advantages	Cheap, deterministic proxy for cost and latency; reveals context bloat.
Limitations	Tokenisers differ between models; prompt caching changes the real cost; fewer tokens ≠ better answers.
Used in (paper / benchmark)	Production practice; context-budget analyses
In the Hitchhiker's Guide	Hitchhiker §18.2.6 Token Counting and Budget Monitoring; §18.8.2 Cost Management; §19.10.2 The Context Budget (TOC verified)

23. Cost (per task / per success)	
Definition	Model cost (input and output token prices) plus tool/API/infrastructure cost. Cost per successful task divides total spend by successes, so cheap-but-failing agents are not rewarded.
Formula	Cost/task = Σ(tokens_in·p_in + tokens_out·p_out) + tool cost; Cost/success = total cost / # successes
Range	0 to ∞ (currency)
Direction	Lower is better
Good range	Below the value of the task (task-dependent)
Bad range	Cost per success above the value or human alternative
Recommended threshold	No universal threshold (task-dependent). Engineering recommendation: pass if cost per successful task is below the value of the task or the human alternative, with headroom for variance.
Example calculation	Illustrative prices (NOT real quotes): $3 per M input, $15 per M output tokens → 12,000·$3/1e6 + 2,000·$15/1e6 = $0.066 per task. 100 tasks cost $6.60; with 55 successes, cost per successful task = $0.120.
Advantages	Business-relevant; cost per success unifies accuracy and efficiency.
Limitations	Prices change; shared infrastructure is hard to attribute; ignores hidden review costs.
Used in (paper / benchmark)	Production practice; cost-controlled evaluation arguments in the agent-benchmark literature
In the Hitchhiker's Guide	Hitchhiker §18.8.2 Cost Management; §19.12 The Economics of Loops; §11.12 Cost Analysis; §27.7.3 Cost Optimization Strategies (TOC verified)

Human evaluation

24. Human Evaluation (ratings & inter-annotator agreement)	
Definition	Annotators rate outputs (Likert), compare pairs or label failures. Quality of the labels is measured with Cohen's κ (two raters) or Fleiss' κ (several).
Formula	Cohen's κ = (p_o − p_e)/(1 − p_e); report mean rating ± CI
Range	κ from −1 to 1; Likert on its scale
Direction	Higher rating / higher κ is better
Good range	κ 0.61–0.80 substantial, >0.80 almost perfect (Landis & Koch)
Bad range	κ below ~0.4 (fair or worse): guidelines or task too ambiguous
Recommended threshold	κ bands are an established convention, not a law of nature. Engineering recommendation: require κ ≥ 0.6 before using labels as ground truth; rating pass thresholds are task-dependent.
Example calculation	Two annotators label 100 outputs, agree on 90 (p_o = 0.90), chance agreement p_e = 0.60 → κ = 0.75 ("substantial" on the Landis–Koch scale). Mean Likert rating, e.g. 4.1/5, is reported together with κ.
Advantages	Gold standard for subjective quality; also used to calibrate LLM judges.
Limitations	Costly and slow; annotator bias and fatigue; low agreement on open-ended tasks.
Used in (paper / benchmark)	Landis & Koch (1977) κ interpretation [from memory]; Chatbot Arena; MT-Bench human study
In the Hitchhiker's Guide	Hitchhiker §14.2.1 Human Annotation Pipelines; §14.2.2 Inter-Annotator Agreement (Cohen's & Fleiss' κ); §14.2.4 Crowdsourcing vs Expert (TOC verified)

4. Final comparison table

All thresholds marked “task-dependent” have no universal value in the literature; “Eng. rec.” marks an engineering recommendation, not a published standard. Source verification status for each entry is in Section 3 and Section 5.

Metric	Definition	Formula	Range	Good Range	Bad Range	Threshold	Higher/Lower Better	Source
Task Success Rate (TSR)	Share of tasks whose end goal is verifiably achieved.	TSR = (# tasks with success check = true) / (# tasks)	0 to 1 (0%–100%)	Close to or above the human baseline on the same benchmark; no universal number	Far below human baseline or below a trivial baseline	Task-dependent. Eng. rec.: lower CI bound above human baseline/SLA	Higher is better	WebArena; τ-bench; SWE-bench
Progress Rate / Partial Goal Completion	Fraction of sub-goals achieved, giving credit for partial progress.	PR = (1/N) Σ_i max_t (matched sub-goals at step t / total sub-goals of task i)	0 to 1	High and close to TSR (if PR ≫ TSR the agent gets close but does not finish)	Low PR: agent stalls early	Task-dependent (no universal value). Eng. rec.: set from risk, baseline or SLA	Higher is better	AgentBoard
Plan Validity / Planning Accuracy	Share of generated plans that are executable and reach the goal.	Plan accuracy = (# valid goal-reaching plans) / (# plans generated)	0 to 1	Task-dependent; classical planning benchmarks have shown low LLM scores without tools	Near 0 on standard planning tasks	Task-dependent (no universal value). Eng. rec.: set from risk, baseline or SLA	Higher is better	PlanBench; ReAct
Tool-Selection Accuracy	How often the agent chooses the correct tool.	Tool accuracy = (# correct tool selections) / (# tool-call decisions)	0 to 1	Task-dependent; higher with fewer, well-described tools	Drops sharply as tool library grows	Task-dependent (no universal value). Eng. rec.: set from risk, baseline or SLA	Higher is better	BFCL; ToolBench
Tool-Call Precision, Recall & F1	Overlap between tools the agent called and tools it should have called.	P = TP/(TP+FP); R = TP/(TP+FN); F1 = 2PR/(P+R)	Each 0 to 1	task-dependent; read P and R separately	Low P = noisy/over-calling; low R = missing required steps	Task-dependent (no universal value). Eng. rec.: set from risk, baseline or SLA	Higher is better	ToolBench-style evals
Argument / Parameter Accuracy	Whether tool-call arguments are correct, not just the tool name.	Param acc = (# correct argument values)/(# required argument values); Call EM = (# fully correct calls)/(# calls)	0 to 1	task-dependent; call-level exact match is the stricter number	High tool accuracy but low argument accuracy signals ‘right tool, wrong inputs’	Task-dependent (no universal value). Eng. rec.: set from risk, baseline or SLA	Higher is better	BFCL
Plan Quality & Plan Adherence	How good the plan is and how closely the agent follows it.	Adherence = (# plan steps executed as planned)/(# plan steps); Optimality = cost(best plan)/cost(agent plan)	0 to 1 (optimality ≤ 1 if best is known)	task-dependent	Low adherence with high success = plan is decorative; high adherence with low success = bad plan	Task-dependent (no universal value). Eng. rec.: set from risk, baseline or SLA	Higher is better	ReAct; plan-execute evals
Trajectory / Process Evaluation	Quality of the sequence of steps, not only the final result.	Step accuracy = (# correct steps)/(# steps); in-order match: reference actions appear as an ordered subsequence	0 to 1	task-dependent	Correct final answers reached by lucky/unsafe paths	Task-dependent (no universal value). Eng. rec.: set from risk, baseline or SLA	Higher is better	AgentBench; PRM literature
Step / Trajectory Efficiency	How many steps the agent needs relative to a reference or optimum.	Efficiency = optimal steps / actual steps (or mean steps per task)	(0, 1] when optimum is known; otherwise 1 to ∞ steps	Near 1.0 with success unchanged (task-dependent)	Much lower than 1: loops, redundant calls	Task-dependent (no universal value). Eng. rec.: set from risk, baseline or SLA	Higher is better (as ratio); lower is better (as step count)	AgentBench; SWE-agent
Pass@k	Probability that at least one of k samples solves the task.	pass@k = E_tasks[1 − C(n−c, k) / C(n, k)]	0 to 1; non-decreasing in k	task-dependent; meaningful only if a verifier can pick the correct sample	Large gap between pass@1 and pass@k means answers are not reliable one-shot	Task-dependent (no universal value). Eng. rec.: set from risk, baseline or SLA	Higher is better	Codex/HumanEval
Pass^k (pass-hat-k)	Probability that all k independent trials of a task succeed.	pass^k = E_tasks[C(c, k) / C(n, k)] (≈ p^k for success prob. p)	0 to 1; non-increasing in k	Stays close to pass^1 as k grows (task-dependent)	Steep fall with k (τ-bench: pass^8 <25% in retail for gpt-4o) [verified]	Task-dependent. Eng. rec.: pick k = expected repeats; 99% per run ≈ 90% at k=10	Higher is better	τ-bench

Metric	Definition	Formula	Range	Good Range	Bad Range	Threshold	Higher/Lower Better	Source
Reliability / Run-to-Run Consistency	Whether repeated runs of the same task give the same outcome.	Consistency = (1/N) Σ_i 1[all k outcomes of task i are equal]	0 to 1	task-dependent; for deterministic decoding should be ≈ 1	Many tasks flip between pass and fail	Task-dependent (no universal value). Eng. rec.: set from risk, baseline or SLA	Higher is better	τ-bench
LLM-as-a-Judge (score and agreement)	Uses a strong LLM to grade outputs; validated by agreement with humans.	Agreement κ = (p_o − p_e)/(1 − p_e); also judge accuracy vs human labels	κ from −1 to 1; scores per rubric scale	κ ≈ 0.61–0.80 or higher against humans (Landis–Koch 'substantial' band)	κ < 0.4, or strong position/verbosity bias	No universal value. Eng. rec.: κ ≥ 0.6 vs humans after calibration	Higher agreement is better	MT-Bench; G-Eval
Elo / Pairwise Preference (Bradley–Terry)	Ranks agents/models from head-to-head human or judge preferences.	E_A = 1/(1+10^((R_B−R_A)/400)); R_A' = R_A + K(S_A − E_A); BT: P(A>B)=σ(s_A−s_B)	Win rate 0–1; Elo/BT scores unbounded (relative)	Higher than rivals with non-overlapping confidence intervals	Overlapping CIs = no demonstrated difference	Relative only. Eng. rec.: win rate >50% with CI excluding 50%	Higher is better	Chatbot Arena
Faithfulness / Groundedness	Share of answer claims supported by the provided context.	Faithfulness = (# claims supported by context)/(# claims in answer)	0 to 1	task-dependent; regulated domains need values near 1	Many unsupported claims	Task-dependent (no universal value). Eng. rec.: set from risk, baseline or SLA	Higher is better	RAGAS
Factual Accuracy (FActScore-style)	Share of atomic facts in the output that are true.	FActScore = (# supported atomic facts)/(# atomic facts)	0 to 1	task-dependent	Low scores on long-form generation	Task-dependent (no universal value). Eng. rec.: set from risk, baseline or SLA	Higher is better	FActScore; TruthfulQA
Context Precision & Context Recall	Whether retrieved context is relevant (precision) and complete (recall).	Context precision = (# relevant retrieved)/(# retrieved); Context recall = (# reference claims supported by context)/(# reference claims)	Each 0 to 1	task-dependent	Low precision wastes context; low recall means the answer cannot be grounded	Task-dependent (no universal value). Eng. rec.: set from risk, baseline or SLA	Higher is better	RAGAS
Hallucination Rate	Share of outputs (or claims) containing unsupported or false content.	Hallucination rate = (# hallucinated responses [or claims])/(# responses [or claims])	0 to 1	task-dependent; near 0 for high-stakes domains	Any non-trivial rate in safety-critical use	Task-dependent (no universal value). Eng. rec.: set from risk, baseline or SLA	Lower is better	HaluEval; SelfCheckGPT
Attack Success Rate (ASR) & Safety Metrics	Share of adversarial attempts (e.g., prompt injection) that succeed.	ASR = (# successful attacks)/(# attack attempts); also utility under attack	0 to 1	As close to 0 as the risk requires (task-dependent)	Any success on high-severity actions	Task-dependent by severity. Eng. rec.: target 0 on irreversible actions	Lower is better (ASR); report utility alongside	AgentDojo
Robustness & Recovery	Performance under perturbations and ability to recover from errors.	Robustness drop = SR_clean − SR_perturbed; Recovery rate = (# recovered failures)/(# injected failures)	Drop: −1 to 1 (typically 0–1); recovery: 0 to 1	Small drop; high recovery (task-dependent)	Large drop under minor rewording (reasoning-collapse behaviour)	Task-dependent (no universal value). Eng. rec.: set from risk, baseline or SLA	Lower drop / higher recovery is better	ToolEmu; AgentDojo
Latency	Wall-clock time to complete a request or task.	Latency = t_end − t_start; report p50, p95, p99	0 to ∞ (seconds)	Within the UX/SLA budget (task-dependent)	Long tail (p95/p99) beyond the budget	Task-dependent. Eng. rec.: set a p95 budget per interaction type	Lower is better	Industry practice
Token Usage	Tokens consumed per task (input + output, incl. reasoning).	Tokens/task = Σ_steps (input tokens + output tokens)	0 to ∞	Low at equal success (task-dependent)	Growth with each step (context re-reads), loops	Task-dependent (no universal value). Eng. rec.: set from risk, baseline or SLA	Lower is better (at equal success)	Industry practice
Cost (per task / per success)	Money spent per task, and per successful task.	Cost/task = Σ(tokens_in·p_in + tokens_out·p_out) + tool cost; Cost/success = total cost / # successes	0 to ∞ (currency)	Below the value of the task (task-dependent)	Cost per success above the value or human alternative	Task-dependent. Eng. rec.: cost/success below task value	Lower is better	Industry practice
Human Evaluation (ratings & inter-annotator agreement)	Human ratings or preferences, with agreement statistics.	Cohen's κ = (p_o − p_e)/(1 − p_e); report mean rating ± CI	κ from −1 to 1; Likert on its scale	κ 0.61–0.80 substantial, >0.80 almost perfect (Landis & Koch)	κ below ~0.4 (fair or worse): guidelines or task too ambiguous	κ bands are a convention; Eng. rec.: κ ≥ 0.6 for ground-truth labels; rating cut-off task-dependent	Higher rating / higher κ is better	Landis–Koch; Arena

5. References and verification status

Primary	Roitman, H. The Hitchhiker’s Guide to Agentic AI: From Foundations to Systems. arXiv:2606.24937 (2026). [abstract + table of contents verified; evaluation-chapter body text not read]
[verified]	Yao, Shinn, Razavi, Narasimhan. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv:2406.12045 (2024). Introduces pass^k; gpt-4o <50% success, pass^8 <25% (retail).
[verified]	Zhou et al. WebArena: A Realistic Web Environment for Building Autonomous Agents. arXiv:2307.13854 (2023). Best GPT-4 agent 14.41% vs human 78.24%.
[verified]	Es, James, Espinosa-Anke, Schockaert. Ragas: Automated Evaluation of Retrieval Augmented Generation. arXiv:2309.15217.
[verified]	Debenedetti et al. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. arXiv:2406.13352. 97 tasks, 629 security test cases.
[from memory]	Chen et al. Evaluating Large Language Models Trained on Code (Codex/HumanEval; pass@k). arXiv:2107.03374.
[from memory]	Jimenez et al. SWE-bench. arXiv:2310.06770. Liu et al. AgentBench. arXiv:2308.03688. Mialon et al. GAIA. arXiv:2311.12983.
[from memory]	Ma et al. AgentBoard. arXiv:2401.13178. Valmeekam et al. PlanBench. arXiv:2206.10498. Yao et al. ReAct. arXiv:2210.03629.
[from memory]	Patil et al. Gorilla / Berkeley Function-Calling Leaderboard. arXiv:2305.15334. Qin et al. ToolLLM/ToolBench. arXiv:2307.16789.
[from memory]	Zheng et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. arXiv:2306.05685. Chiang et al. Chatbot Arena. arXiv:2403.04132. Liu et al. G-Eval. arXiv:2303.16634.
[from memory]	Min et al. FActScore. arXiv:2305.14251. Lin et al. TruthfulQA. arXiv:2109.07958. Li et al. HaluEval. arXiv:2305.11747. Manakul et al. SelfCheckGPT. arXiv:2303.08896.
[from memory]	Zhan et al. InjecAgent. arXiv:2403.02691. Mazeika et al. HarmBench. arXiv:2402.04249. Ruan et al. ToolEmu. arXiv:2309.15817. Landis & Koch (1977), Biometrics 33(1).

To close the main gap: open the Hitchhiker’s Guide PDF at §14.6 and §30.22, and compare its definitions and any thresholds with the entries above. Where they differ, the guide’s wording should be added as a citation in the “In the Hitchhiker’s Guide” row.